

2026 超专业级别硬件能力认证第一轮
(C5P-S1) 提高级 C++ 语言试题 参考答案与解析

认证时间：2026 年 9 月 18 日 14:30~16:30

编者：唐一潇、季灵、裘泽燿

考生注意事项：

- 试题纸共有 17 页，答题纸共有 1 页，满分 100 分。请在答题纸上作答，写在试题纸上的一律无效。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

说明：本文件对应自制模拟卷，不代表任何官方机构。题面、代码、解析和考点矩阵按本套 42 题版本同步。

分值与题号

部分	题号	题量与计分	小计
单项选择	1-15	15×2 分	30 分
阅读程序（1）字符串边界处理	16-20	$3 \times 1.5 + 1 \times 3 + 1 \times 4$ 分	11.5 分
阅读程序（2）状态压缩 DP	21-26	$2 \times 1.5 + 1 \times 2 + 3 \times 3$ 分	14 分
阅读程序（3）区间乘加线段树	27-32	$3 \times 1.5 + 2 \times 3 + 1 \times 4$ 分	14.5 分
完善程序	33-42	10×3 分	30 分
总分			100 分

一、单项选择题（1-15）

1. 答案：B

解析：cat 的含义是 concatenate，会按给定顺序读取并输出文件内容，因此用于连接并显示文件。其余选项中，cp 用于复制，ping 用于测试网络连通性，touch 用于创建文件或更新时间戳。

2. 答案：C

解析：二叉树中每个非叶结点有两个孩子，因此含 m 个叶子的满二叉树有 $m - 1$ 个非叶结点，共 $2m - 1$ 个结点。每个结点有两个指针域，指针域总数为 $2(2m - 1) = 4m - 2$ ；树的边数为结点数减一，即 $2m - 2$ ，这些边对应非空指针，所以空指针域为 $(4m - 2) - (2m - 2) = 2m$ 。

3. 答案：B

解析：高度为 h 的完全 5 叉树的叶子数是 $\Theta(5^h)$ 。从一个叶子向根访问的路径包含 $h + 1$ 个结点，数量级为 $\Theta(h)$ ；题目对每个叶子都做一次这样的访问，故总次数是 $\Theta(5^h) \cdot \Theta(h) = \Theta(h5^h)$ 。

4. 答案：D

解析：不考虑限制时，5 次向右和 5 次向上共有 $\binom{10}{5} = 252$ 条路径。违反 $x \geq y$ 的路径在第一次越过对角线后可用反射法计数为 $\binom{10}{4} = 210$ 条，因此合法路径数为 $252 - 210 = 42$ ，即卡特兰数 C_5 。

5. 答案: A

解析: 栈 A 中先入队的元素在栈底, 栈 B 中先转入的元素会位于栈顶。B 非空时, 栈顶仍是队列中最早的元素, 直接弹出即可; 只有 B 为空时才需要把 A 全部转移到 B。若每次都转移, 原有 B 中的元素顺序会被破坏。

6. 答案: C

解析: 设选出的数为 $x_1 < x_2 < \cdots < x_6$, 相邻限制为 $x_i + 1 < x_{i+1}$ 。令 $y_i = x_i - (i - 1)$, 则 $1 \leq y_1 < \cdots < y_6 \leq 15$, 而且这个变换可逆, 所以问题等价于从 15 个位置中任选 6 个, 方案数为 $\binom{15}{6} = 5005$ 。

7. 答案: B

解析: vector 在头部插入或删除时通常要移动后续元素, 复杂度不是 $O(1)$; deque 的随机访问为 $O(1)$, 两端插入和删除也支持均摊 $O(1)$ 。set 的键必须互异, 允许重复键的是 multiset; priority_queue::top() 只返回堆顶引用, 删除元素应调用 pop(), 因此只有 B 正确。

8. 答案: C

解析: 拓扑序的定义是: 对每条边 $u \rightarrow v$, u 必须排在 v 前面, 所以 C 直接符合定义。DAG 的拓扑序通常不唯一; 若图中存在有向环, 环上每个点都要求排在下一个点前, 无法形成线性序列, 因此 A、B、D 均不对。

9. 答案: C

解析: 文本串的长度为 11, 模式串 ababa 长度为 5。逐个检查可知, 匹配起点为 0、2、4、6; 起点 8 时还需要访问到下标 12, 已经超出文本串范围。题目允许匹配重叠, 因此总和为 $0 + 2 + 4 + 6 = 12$ 。

10. 答案: B

解析: SPFA 在最坏情况下可能反复松弛, 复杂度可达 $O(nm)$, 所以 A 错。Dijkstra 的正确性需要非负边权, 使用邻接表和二叉堆时复杂度为 $O((n + m) \log n)$, B 对; Bellman-Ford 可以通过额外一次松弛判断源点可达的负环, C 错; Floyd-Warshall 三重枚举中间点和两个端点, 复杂度是 $O(n^3)$, D 错。

11. 答案: D

解析: 令 $p = 20260913$ 。在模 p 的非零剩余类中, k 与 $p - k$ 互为相反数, 故 $\frac{1}{k} + \frac{1}{p-k} = 0$; 把 $k = 2, \dots, p-1$ 两两配对后, 括号内的和等于 $\sum_{k=1}^{p-1} 1/k - 1 = -1$ 。由于 p 是奇数, $(-1)^p = -1 \equiv p-1 = 20260912 \pmod{p}$, 所以选 D。

12. 答案: B

解析: 递归式对应 $a = 7, b = 3$, 临界项为 $n^{\log_3 7}$, 且 $\log_3 7 \approx 1.771$ 。非递归项为 $\Theta(n \log n)$, 指数部分只有 n^1 , 比临界项低一个多项式量级, 满足主定理第一种情况; 因此 $T(n) = \Theta(n^{\log_3 7})$, 选 B。

13. 答案: B

解析: 共有 $\binom{9}{2} = 36$ 种删边方案。边 $4 \rightarrow 5$ 和 $7 \rightarrow 8$ 都是必经边, 删去其中任一条的方案有 $8 + 8 - 1 = 15$ 种; 在不删这两条边时, 还需排除使 1 到 4 断开的方案: 从路径 $1 \rightarrow 2 \rightarrow 4$ 和 $1 \rightarrow 3 \rightarrow 4$ 各删一条, 共 $2 \times 2 = 4$ 种; 使 5 到 7 断开的方案为删去 $5 \rightarrow 7$, 再删去 $5 \rightarrow 6$ 或 $6 \rightarrow 7$, 共 2 种。因此合法方案为 $\binom{7}{2} - 4 - 2 = 15$, 答案为 B。

14. 答案: B

解析: 用容斥原理: 能被 2、3、7 分别整除的数有 100、66、28 个; 两两同时整除的有 $\lfloor 200/6 \rfloor = 33$ 、 $\lfloor 200/14 \rfloor = 14$ 、 $\lfloor 200/21 \rfloor = 9$ 个; 三者同时整除的有 $\lfloor 200/42 \rfloor = 4$ 个。因此不能被任何一个整除的数为 $200 - 100 - 66 - 28 + 33 + 14 + 9 - 4 = 58$ 。

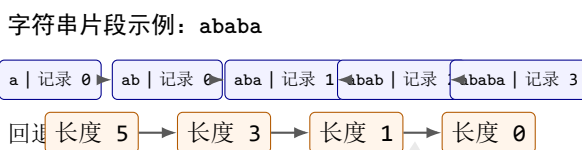
15. 答案: A

解析: 由递推式可先得到已知公式 $A(3, n) = 2^{n+3} - 3$, 所以 $A(3, 3) = 2^6 - 3 = 61$ 。模 101 计算第二项时, 指数只需模 100; $10^{18} + 3 \equiv 3 \pmod{100}$, 且 $2^{50} \equiv -1 \pmod{101}$, 从而 $2^{10^{18}+3} \equiv 2^3 = 8$ 。故第二项为 $8 - 3 = 5$, 二元组为 (61, 5)。

二、阅读程序 (16-32)

(1) 字符串边界处理

程序总析: 程序逐个处理字符串位置, 为每个位置记录一个可继续比较的长度; 遇到不匹配时沿已记录的长度逐级回退, 最后把完整长度和各级相同首尾片段的长度相加。



16. 答案: A (正确)

解析: 程序改用指针后, mark 仍记录每个位置对应的长度。处理 ababa 时记录值依次为 0, 0, 1, 2, 3; 最后一个字符 a 与长度为 2 的候选片段后继匹配, 所以 $*(mark+4)=3$, 判断正确。

17. 答案: C

解析: 第 34 行第一次取 len=5, 第 35 行累加 5; 随后令 len=fetch(5)=3, 再累加 3; 接着令 len=fetch(3)=1, 累加 1; 最后 fetch(1)=0, 循环结束。因此输出为 $5 + 3 + 1 = 9$, 选 C。

18. 答案: A

解析: 输入 bbba 时, 处理最后一个 a, 初始候选长度为 fetch(3)=2。第 17 行的 while 会连续回退到 1、再回退到 0, 最终写入 $*(mark+3)=0$, 累加循环只得到 4。若改成 if, 只回退一次到 1, 随后比较仍失败却停下, 写入 $*(mark+3)=1$, 回退链为 $4 \rightarrow 1 \rightarrow 0$, 输出 5, 所以二元组为 (4, 5)。

19. 答案: C

解析: 该串长度为 14, 逐级回退得到的长度依次为 14、8、2、0; 其中长度 8 和长度 2 分别对应相同的首段与尾段。第 34-35 行把非零长度相加, 得到 $14 + 8 + 2 = 24$, 因此选 C。

20. 答案: B

解析: 第 16-21 行通过辅助函数和指针访问完成逐位置处理; 每次匹配成功会使候选长度增加, 而第 17-19 行的回退会使它严格减小, 因此所有增加和回退的总次数为 $O(n)$ 。第 34-35 行沿记录的长度逐级回退, 总次数也不超过 n , 所以时间复杂度为 $O(n)$; 动态申请的字符缓冲区和整数数组占用 $O(n)$ 空间, 末尾用 delete[] 释放, 选 B。

(2) 状态压缩 DP

程序总析: 每一行用无相邻 1 的位掩码表示选取位置, 转移要求相邻两行掩码不相交。



21. 答案: A (正确)

解析: 三列掩码从 000 到 111 枚举。条件 $(s \& (s \ll 1)) == 0$ 会排除含相邻 1 的 011, 110, 111, 保留下来的正好是 000, 001, 010, 100, 101, 所以 `st.size()` 为 5。

22. 答案: A (正确)

解析: 这里 $s=101$ 表示当前行取第 0、2 列, $t=001$ 表示上一行取第 0 列。按位与为 $101 \& 001 = 001$, 结果非零, 程序立即执行 `continue`, 不会执行后面的 `f[cur][s] += f[pre][t]`, 所以判断正确。

23. 答案: A

解析: 当 $n=3, m=2$ 且没有禁用格时, 合法行状态为 00, 01, 10。从初始状态出发, 三行的方案数依次为 3、7、17。若删除第 26 行, 当前滚动数组没有清零, 上一轮的旧值会混入本轮转移, 结果变为 23, 因此二元组为 (17, 23)。

24. 答案: B

解析: 预处理阶段扫描全部 2^m 个掩码, 耗时 $O(2^m)$ 。设合法状态数为 $V = \text{st.size}()$, 每一行都枚举当前状态和上一行状态各一次, 耗时 $O(V^2)$; 共 n 行, 所以总复杂度为 $O(2^m + nV^2)$, 即选 B。

25. 答案: C

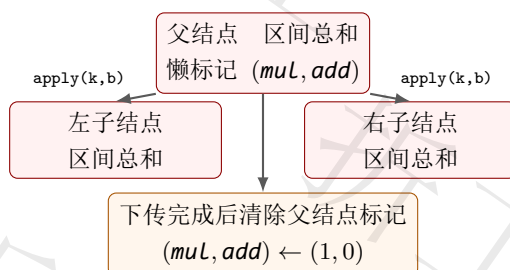
解析: 三列时合法状态按顺序为 000, 001, 010, 100, 101。第一行每个状态的方案数都为 1; 第二行各状态可接的上一行状态数分别为 5、3、4、3、2, 总数为 $5 + 3 + 4 + 3 + 2 = 17$, 因此选 C。

26. 答案: A

解析: 滚动数组在第 i 行通过 `pre` 和 `cur` 交替使用, 先清空当前层, 再从上一层转移; 保留第 23-24 行时三行两列的答案为 17。删除交换语句后, 第一轮会把含有初始值的数组清空, 后续每层都从全零数组转移, 最终答案为 0, 所以为 (17, 0)。

(3) 区间乘加线段树

程序总析: 每个结点保存所代表区间的总和。一次完整覆盖的更新可以先记在当前结点, 之后访问子区间时再传给两个子结点。



27. 答案: A (正确)

解析: 当 $ql \leq l$ 且 $r \leq qr$ 时, 当前结点区间被完整覆盖。第 39 行调用一次 `apply`, 把更新合并进当前结点的总和和懒标记; 第 40 行立即返回, 因此不会继续访问子结点。

28. 答案: B (错误)

解析: 第 32、33 行把父结点当前的 `mul` 和 `add` 下传给左右子结点, 第 34、35 行再把父结点标记恢复为恒等变换 $(1, 0)$ 。若删除第 35 行, 旧的加法标记会在父结点下一次下传时再次应用, 导致部分区间重复加法, 输出可能改变。

29. 答案: A (正确)

解析: 建树递归每个结点一次, 耗时 $O(n)$ 。懒标记保证一次区间更新或查询只访问 $O(\log n)$ 层上的结点, 故 q 次操作耗时 $O(q \log n)$; 总复杂度是 $O(n + q \log n)$, 在通常的 $n \geq 2$ 时属于 $O((n + q) \log n)$ 。

30. 答案: B

解析: 设当前区间长度为 $L = r - l + 1$, 原区间总和为 `tr[p].sum`。更新把每个元素 x 变为 $kx + b$, 所以总和变为 $\sum(kx + b) = k \sum x + Lb$, 对应代码中的 `tr[p].sum*k+(r-l+1)*b`, 最后整体对 P 取模。

31. 答案: A

解析: 第 34 行把父结点的乘法标记重置为 1, 第 35 行把加法标记重置为 0; 若删除或提前执行任一重置, 旧标记就可能丢失或重复下传。第 32、33 行通过结点编号 $p \ll 1$ 和 $p \ll 1 | 1$, 分别给左右子结点应用同一组标记; 交换这两次传递的顺序不改变两个子区间得到的值, 因此选 A。

32. 答案: B (4 分)

解析: 逐次执行区间乘加更新: 初始数组为 2 1 3 5 4 6; 第一步对第 2-6 项做 $2x + 1$, 得到 2 3 7 11 9 13; 第二步对第 1-4 项做 $3x + 2$, 得到 8 11 23 35 9 13; 第三步对第 3-5 项做 $x + 4$, 得到 8 11 27 39 13 13。最后查询区间 $[2, 6]$, 结果为 $11 + 27 + 39 + 13 + 13 = 103$, 选 B。

三、完善程序 (33-42)

(1) 三色序列

33. 答案: C

解析: `pre[t][i]` 表示前 i 个字符中颜色 t 的出现次数。处理位置 i 时, 应先继承前 $i - 1$ 个字符的计数; 若当前位置颜色 c 恰好等于 t , 再增加 1, 所以填 `pre[t][i-1] + (t == c)`。

34. 答案: B

解析: 状态 `f[0][0][0][3]` 表示还没有放置任何字符, `last=3` 只作为“没有上一个颜色”的特殊标记。此时没有发生相邻交换, 最小代价应为 0; `INF` 表示不可达, 不能作为初始值。

35. 答案: A

解析: 循环处理长度为 `len` 的前缀, 若其中使用了 r 个 R 和 g 个 G, 剩余位置只能由 Y 填充。因此已使用的 Y 数量为 $y = \text{len} - r - g$; 若超出 0 到 `cnt[2]` 的范围, 状态立即被跳过。

36. 答案: A

解析: 加入颜色 c 前必须同时满足两个条件: 它不能与上一个颜色 `last` 相同, 并且该颜色仍有剩余数量。代码在条件为真时执行 `continue`, 所以“相同”或“已用完”任一情况都要跳过, 填 `c == last || used[c] == cnt[c]`。

37. 答案: C

解析: `val` 是已经构造当前前缀所需的最少交换次数, `w` 是把新字符移到当前位置产生的新增交换次数。加入该字符后, 新状态的代价就是两者之和 `val + w`, 再用 `min` 保留同一状态的最优值。

(2) 盒子

38. 答案: B

解析: lo 维护最终要保留的 need 个较小间隙, hi 保存其余间隙。若 lo 数量超出 need, 应删除其中最大的候选, 才能保持总和最小; prev(lo.end()) 正是 lo 的最大元素。

39. 答案: B

解析: x 指向 lo 中最大的间隙, y 指向 hi 中最小的间隙。若 $*x \leq *y$, 交换它们不会使总和下降; 只有 $*x > *y$ 时, 用 y 替换 x 才能减小答案, 因此循环在 $*x \leq *y$ 时停止。

40. 答案: B

解析: 交换前答案包含 $vx=*x$, 交换后改为包含 $vy=*y$, 所以答案增量为 $vy-vx$ 。随后把 vx 放回 hi, 把 vy 放入 lo, 集合划分和总和保持一致, 故填 $vy-vx$ 。

41. 答案: A

解析: 按体积排序后, 保留一个间隙就表示相邻的两个盒子属于同一条嵌套链。若最终有 k 条链, 每条链中盒子数减一的间隙被保留, 总数为 $(n_1 - 1) + \dots + (n_k - 1) = n - k$, 所以 $need=n-k$ 。

42. 答案: C

解析: 删除体积为 v 的盒子时, 若它左右都有邻居, 原来相邻的两个间隙分别是 $v-l$ 和 $r-v$; 删除后左右盒子直接相邻, 合并后的间隙为 $(v-l) + (r-v) = r-l$ 。因此第 ⑤ 空应填 $r-l$ 。